Checkmate+ COMP6030 report

Ainan Cawley
acc59@kent.ac.uk

James Ledger
jhl28@kent.ac.uk

Luke Williams
lw571@kent.ac.uk

elendil7



School of Computing
University of Kent

Word Count: 5665

October 13, 2026

Abstract

A report describing the design and implementation of Checkmate+, an online chess platform developed for COMP6030, along with an investigation into how its features affect user enjoyment.

1 Introduction

Chess is a centuries old board game that has adapted well to the internet age. *Chess.com*, the largest online chess platform, has over ten million games played on it every day (Chess.com, 2025).

The online nature of these platforms enable features that don't exist when playing chess with a physical board and pieces. *Lichess.org* introduced voice recognition to make moves in 2023 (Lichess.org, 2023), and *Chess.com* offers alternative animations when moving or capturing pieces. However, there is little published evidence on whether such features improve user enjoyment.

The goal of our project is two-fold. Firstly, to create a website for playing chess that implements three features:

- Computer opponents that have human-like move times.
- Alternative move and capture animations
- Voice-command move input

Secondly, to investigate whether these features affect user enjoyment through surveying and conversational user evaluation sessions.

To do this, we created *Checkmate+*, a site accessible at checkmateplus.org.

2 Project architecture

2.1 Why design and build a microservices architecture?

When designing Checkmate+, we chose a microservices architecture over a monolith. Several factors affected this decision. First, live chess matches are stateful: they are held in the memory of the Game Server for the duration of the match. This is different from a stateless service like authentication, where any pod can serve any request because no per-request state lives on the pod. Second, the post-game analysis engine wraps a native Stockfish binary, which was simpler to isolate in its own service than to embed inside an HTTP-serving Spring Boot process. Third, splitting the system into smaller deployable components lets us redeploy a single service after a bug fix without redeploying the whole platform, and contains the blast radius of a crash to a single pod. Kubernetes restarts the failed pod while the rest of the system is unaffected.

This approach comes with its costs. Every call inside the cluster is a point of failure with its own network semantics, intra-cluster networking must be considered, `.proto` contracts must be kept in sync across services, and the deployment topology is much more complex than a single monolith. Given that the project aims to build a chess

platform comparable in operational shape to Chess.com or Lichess, we judged these costs acceptable and chose to build with scale in mind using the tools enterprises use to run systems of this kind.

2.2 Overview

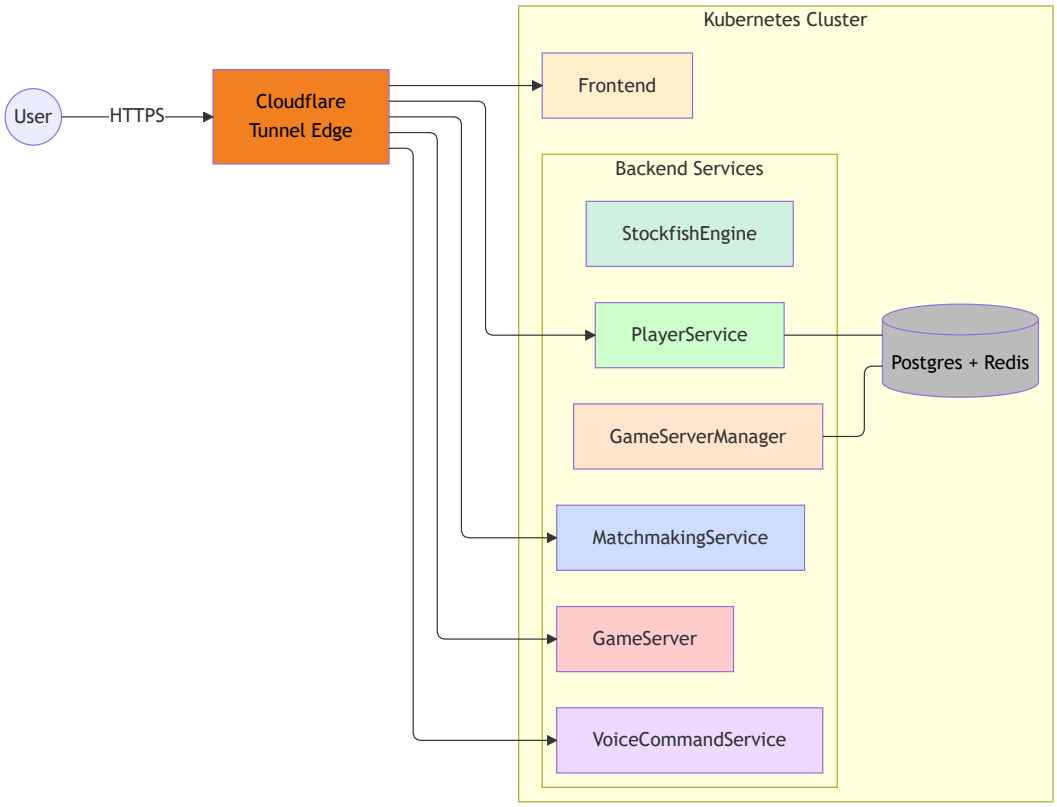


Figure 1: High-level overview of the Checkmate+ platform.

1. **The Game Server (GS).** Responsible for holding live game states; players are connected to a Game Server pod for the duration of a match.

2. **The Game Server Manager (GSM).** Responsible for keeping track of active matches and balancing match allocation across Game Server pods. Tracks match lifecycle in Redis.

3. **The Matchmaking Service (MS).** Responsible for player requests to join matchmaking. Asks the GSM for a free server and tells the client where to connect over STOMP.

4. **The Player Service (PS).** Responsible for the main user-facing API, owns authentication and the database storing player data. Issues JWTs that every other backend service validates.

5. **The Stockfish Engine (SE).** Responsible for analysing completed matches using the Stockfish chess engine and returning per-move evaluations to the Player Service.

6. **The Voice Command Service (VC).** Responsible for the voice command feature, letting users use their voice to make moves.

7. **The Chess Library.** Not a service, but a shared Kotlin library used across the backend. Contains the shared types, the chess engine implementation, and the human-like move time model.

2.3 Inter-service communication

Checkmate+ uses four different communication mechanisms. Each was chosen based on the requirements of the interaction it serves, rather than for consistency across the system. The frontend talks to the backend over three protocols (Figure 2), and the backend services talk among themselves using gRPC (Figure 3).

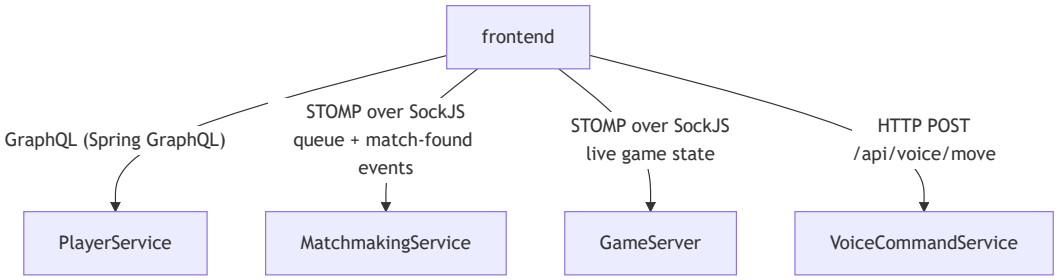


Figure 2: Protocols used between the frontend and the backend services.

Between the browser and the backend, the choice of protocol follows the shape of the data being exchanged. Requests against the Player Service use GraphQL, served by Spring's GraphQL starter, because the frontend often needs slightly different shapes of player profile or match history data on different pages, and a single GraphQL endpoint lets the client pick the fields it needs without the backend exposing a new REST route every time. The Matchmaking Service and the live Game Server both use STOMP over SockJS rather than plain WebSocket, because each interaction is bidirectional and pub/sub in nature: the client subscribes to a "match found" topic on the Matchmaking Service while it waits in the queue, and subscribes to a per-match topic on the Game Server during a live game so that the opponent's moves can be pushed without the client polling. STOMP gives us a frame format and subscription model on top of the raw WebSocket transport. Finally, the Voice Command Service exposes a plain HTTP POST endpoint, because voice input is a one-shot request-response with no need for a persistent connection.

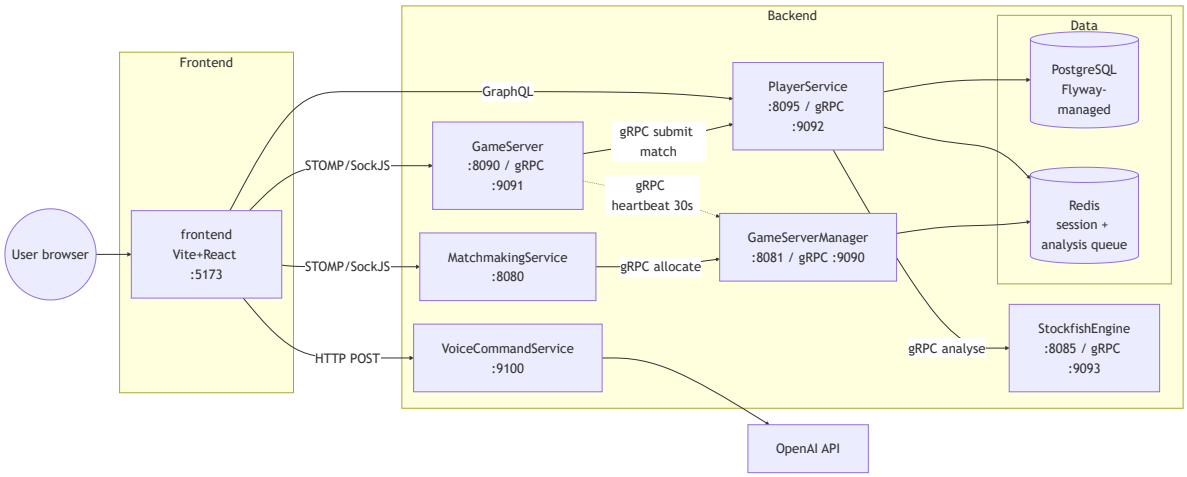


Figure 3: Service topology of the Checkmate+ backend, showing gRPC links between services and dependencies on PostgreSQL, Redis, and the OpenAI API.

Backend services communicate with each other over gRPC. The Matchmaking Service asks the Game Server Manager for a free Game Server, every Game Server sends a heartbeat to the Game Server Manager every 30 seconds, the Game Server reports completed matches to the Player Service, and the Player Service sends matches to the Stockfish Engine for analysis. gRPC was chosen for these calls because the contracts are stable and well-defined, and gRPC's protocol buffer schemas make the contract explicit and the wire format compact compared to JSON over HTTP.

One cost of this approach is that the gRPC contracts live as duplicated `.proto` files inside each service that needs them, rather than as a single shared module. A change to a service's gRPC API therefore requires updating every other service that consumes it, and a missed update will compile cleanly but fail at runtime with a message mismatch.

All backend services run on the JVM as Spring Boot applications written in Kotlin, which gives us a single tooling story across services even though each is a separate deployable.

2.4 Authentication

All authenticated calls between the frontend and the backend, and between the backend services themselves, are authorised using a JSON Web Token (JWT). Figure 4 shows the arrangement. The Player Service issues a JWT after a successful user login (the login flow itself is covered in Section 4.2). Every other backend service runs Spring Security as an OAuth2 resource server and validates that token on each incoming request. The token is supplied as a bearer header on HTTP and GraphQL requests, and on the initial `CONNECT` frame of STOMP sessions.

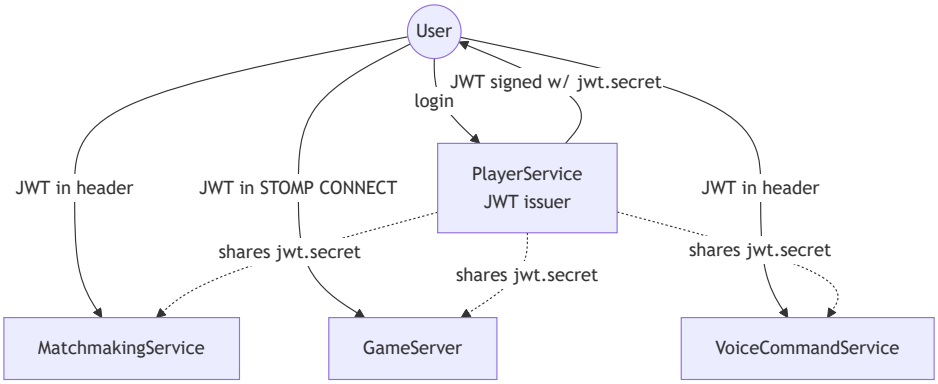


Figure 4: The Player Service issues JWTs; every other backend service validates them using a shared HMAC secret.

The tokens are signed with HMAC-SHA256 using a symmetric secret (`jwt.secret`) that is shared across all backend services as an environment variable. This is simpler than running a separate identity provider and avoids an additional service in the dependency graph, at the cost of two trade-offs. First, every service must be deployed with the same secret value; a mismatch will silently 401 every cross-service call. Second, the secret cannot be rotated without redeploying all services together, so a leak of the secret would require coordinated remediation across the system. These drawbacks were judged to be acceptable for the scale of the project. A production deployment at higher scale would likely benefit from an asymmetric signing scheme (RS256) where only the issuer holds the signing key.

2.5 Match lifecycle: a worked example

To highlight the role of each service, this section traces a single ranked match from the user pressing “Find game” to the post-game analysis being available. Figure 5 shows the sequence.

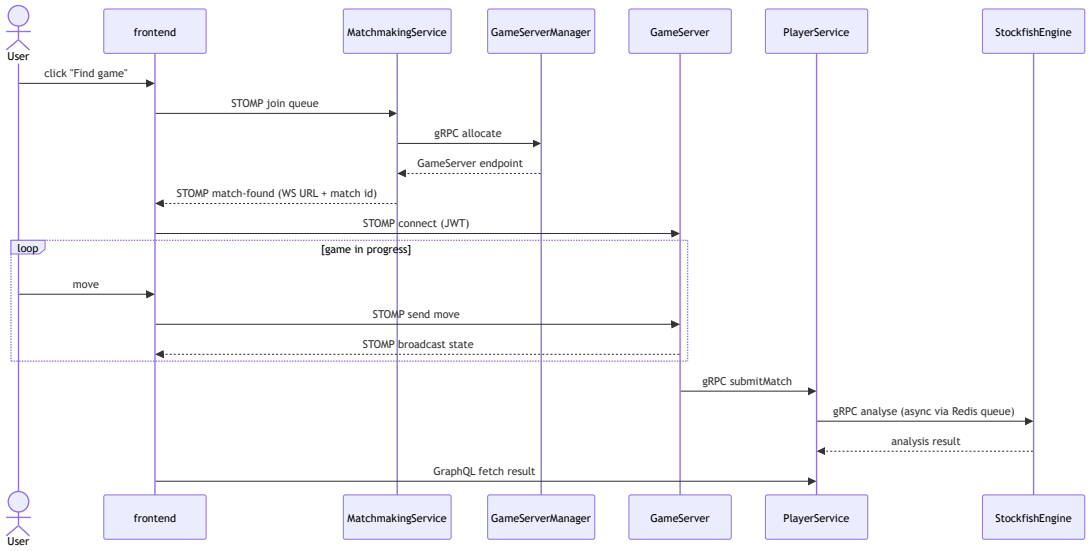


Figure 5: Sequence of interactions during a ranked match, from queue entry to analysis being available.

When the user clicks "Find game", the frontend opens a STOMP connection to the Matchmaking Service and joins the queue. The Matchmaking Service asks the Game Server Manager, over gRPC, for a Game Server with capacity. The Game Server Manager replies with the endpoint of an allocated Game Server pod, and the Matchmaking Service publishes a "match found" message to the user's STOMP topic containing the WebSocket URL and a match identifier.

The frontend then opens a second STOMP connection, this time to the allocated Game Server, presenting the user's JWT on the `CONNECT` frame. From this point on, all moves are pushed by the client into the Game Server and the resulting board state is broadcast to both players over their subscribed topic. The Game Server holds the authoritative game state in memory for the duration of the match.

When the match ends, the Game Server submits the completed match to the Player Service over gRPC. The Player Service persists the match to PostgreSQL and enqueues an analysis job for the Stockfish Engine through Redis on player request. The Stockfish Engine consumes the job, analyses each move, and returns the evaluations back to the Player Service. Some time later, when the user opens the match in their history, the frontend fetches the analysis with a GraphQL query against the Player Service.

This single example utilises all four communication mechanisms described in Section 2.3 and every backend service in the platform.

2.6 Deployment

Checkmate+ is deployed to a Kubernetes cluster as a single Helm chart, with everything described above running in one namespace. The deployment is split across two figures: Figure 6 shows how public traffic enters the cluster and the services it reaches, and Figure 7 shows the internal services and stateful subcharts together with the gRPC links between them.

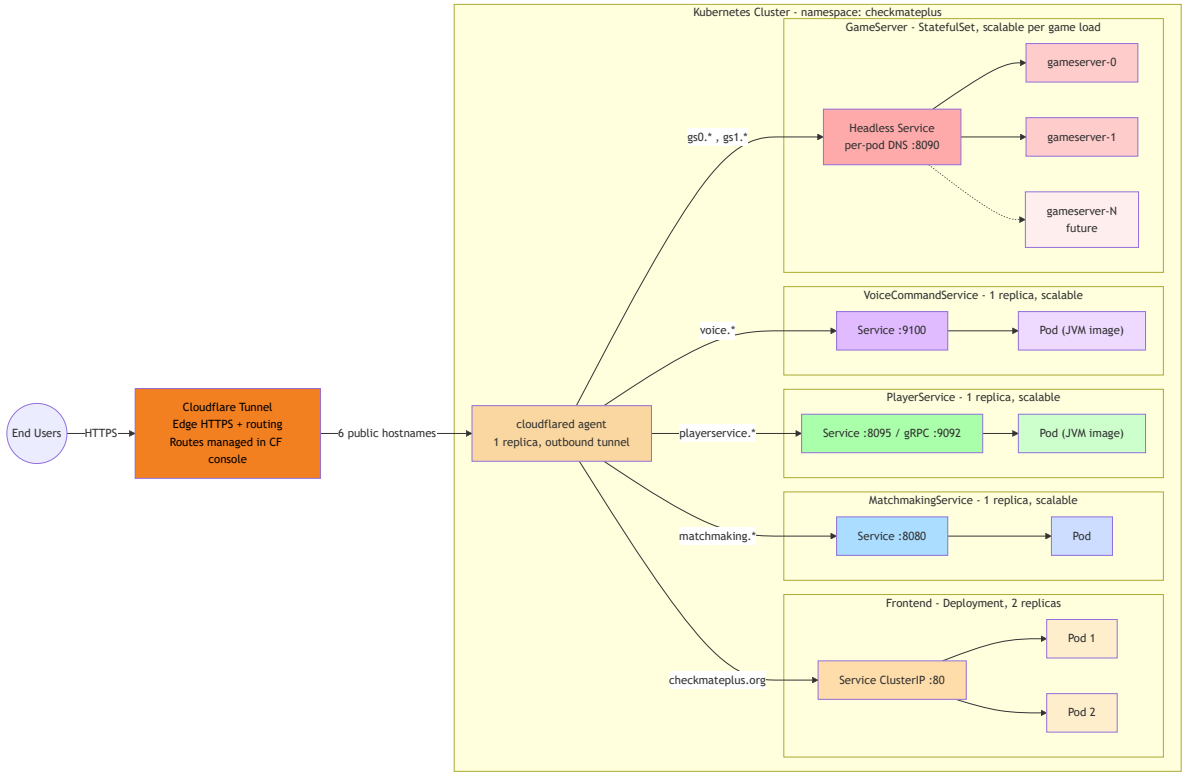


Figure 6: Ingress path and publicly reachable services. Public traffic enters via Cloudflare and is forwarded through the in-cluster `cloudflared` agent to a per-service ClusterIP, which in turn fronts one or more pods.

Ingress via Cloudflare Tunnel. Rather than using a traditional Kubernetes ingress controller with a public load balancer, the cluster runs a single `cloudflared` agent pod which opens an outbound tunnel to the Cloudflare edge. Public traffic for Checkmate+ enters Cloudflare's edge and is routed back through that tunnel to a ClusterIP Service inside the cluster. This avoids exposing any inbound ports on the cluster's host and offloads TLS termination, DDoS protection, and DNS to Cloudflare. The routing rules (six public hostnames such as `checkmateplus.org` for the frontend, `playerservice.checkmateplus.org` for the GraphQL API, and `gs0.checkmateplus.org` / `gs1.checkmateplus.org` for the individual Game Server pods) are configured in the Cloudflare Zero Trust dashboard rather than in the Helm values, which simplifies route changes but means the dashboard

controls what is publicly available.

StatefulSet for the Game Server. The Game Server is deployed as a Kubernetes StatefulSet rather than a Deployment, fronted by a headless Service. This gives each Game Server pod a stable per-pod DNS name (`gameserver-0.checkmateplus-gameserver`, `gameserver-1.checkmateplus-gameserver`, and so on), which is required for the Game Server Manager to route a specific match to a specific pod. A regular Deployment with a round-robin Service would send the player to whichever Game Server pod the load balancer picked, which would not necessarily be the pod holding the game state for their match.

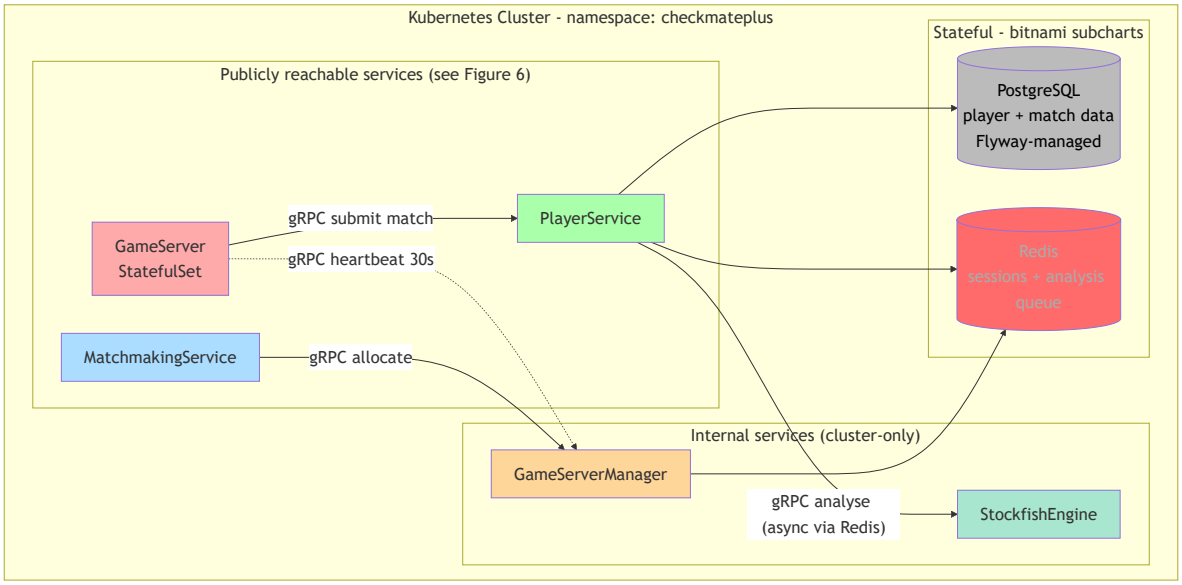


Figure 7: Internal services and stateful subcharts. The Game Server Manager and the Stockfish Engine are not exposed publicly; PostgreSQL and Redis are deployed via Bitnami's Helm subcharts inside the same release.

Stateful dependencies. PostgreSQL and Redis are deployed via Bitnami's Helm subcharts inside the same chart. PostgreSQL holds the player and match data and is managed by Flyway migrations shipped inside the Player Service. Redis carries session data and the post-game analysis queue between the Player Service and the Stockfish Engine.

Container images. Production images are built using Spring Boot's `bootBuildImage` task with the Paketo Buildpacks, producing GraalVM native images compiled with `-O3` and tuned with G1 GC. Native images give significantly faster startup and lower memory footprint than equivalent JVM images, which matters when running multiple Spring Boot services in one cluster. Two services, the Player Service and the Voice Command Service, currently ship as JVM images instead of native ones due to a regression in their native

builds that has not yet been resolved; this is reflected in their image tags (e.g. `2.2.2-jvm`) and in the resource requests in the Helm values.

GitOps with Flux CD. The Helm release itself is reconciled into the cluster by Flux CD, which watches the project's Git repository and applies any changes to the chart or values file automatically. Secrets, including the shared `jwt.secret` discussed in Section 2.4, are stored as Sealed Secrets in the same repository, encrypted to the cluster's public key so that they can be checked in safely.

3 The computer opponent

Online chess platforms often offer the option to play against a computer opponent (also called a "bot"). The moves that a bot makes is determined by a program called a "chess engine". For *Checkmate+*, we created our own chess engine, covered in Section 3.1.

Another aspect to the experience of playing against a bot is how long they take to make their moves. For *Checkmate+*, we implement a feature that neither *Chess.com* or *Lichess.org* have: bots that have human-like move times. This feature will be covered in Section 3.2.

3.1 Chess engine

Our chess engine work can be found in `/ChessLibrary/src/main/kotlin/org/checkmateplus/chessLibrary/chessEngine/Board.kt`.

3.1.1 Background Research

Before implementing the engine, we carried out background research into common chess engine designs. Most engines are built around four main components: (Chessprogramming Wiki, 2024a)

- A board representation to store the current position.
- A move generator to produce possible moves.
- An evaluation function to score positions.
- A search algorithm to choose the best move.

This research helped us to decide on a square-based board representation and a traditional minimax search with alpha-beta pruning, which were both suitable for a project of this scale.

3.1.2 Move Generation

Move generation is the process of producing all possible moves from a given board position, and is foundational for search. Our implementation of move generation has two stages:

The first stage, implemented in `generateAllMoves()`, produces pseudo-legal moves for all pieces belonging to the side to move (black/white). This generates candidate moves for each piece type - pawns, knights, bishops, rooks, queens and kings - based purely on their movement rules, without checking whether the resulting position leaves the moving side's king in check. Each piece type has its own generation function, making the rules for each piece clear and easy to maintain independently.

The second stage, implemented in `generateLegalMoves()`, filters these pseudo-legal moves to produce only fully legal moves. Each candidate move is applied to a copy of the board, and any move that left the moving side's king in check is discarded using the `isInCheck()` function. This ensures that the search algorithm only considers legal positions. Special rules including castling, en passant, and pawn promotion are all handled explicitly within the individual piece generation functions.

3.1.3 Search Algorithm

The search algorithm is responsible for choosing the best move from a given position by looking ahead at possible future game states. We implemented a minimax algorithm with alpha-beta pruning, which is the standard approach for classical chess engines (Chessprogramming Wiki, 2024b).

Minimax works by modelling the game as a tree of positions. White aims to maximise the evaluation score while black aims to minimise it. At each node, the algorithm applies a move to produce a new board state and recursively evaluates the resulting position. When the search reaches its maximum depth, the position is scored using the evaluation function. The best move is then the one that leads to the highest score for white, or the lowest score for black.

To improve efficiency, we implemented alpha-beta pruning. This technique maintains two values - alpha, the best score white can guarantee, and beta, the best score black can guarantee. If a move is found that is worse than the current best for the side to move, that branch is cut off and not explored further. This significantly reduces the number of positions the engine needs to evaluate without affecting the result. Alpha-beta pruning is most efficient when the best moves are searched first, as there'd be more cutoffs (Vrzina, 2023). The single change of prioritising moves where a less valuable piece captures a more valuable piece improved search time by a factor of 3.67 (see `/documentation/testing/Improvingenginespeed.md`).

We also implemented iterative deepening, where the engine performs searches at increasing depths before reaching the target depth. Counter-intuitively, this improves overall performance because earlier searches populate the transposition table - a cache of previously evaluated positions - which helps prune the deeper search more effectively. Move ordering is applied at each node using the transposition table's best move and captures of valuable pieces, further improving the efficiency of alpha-beta pruning.

3.1.4 Evaluation Function

The evaluation function scores a board position from White's perspective, returning a positive value if White is better and a negative if Black is better. This score is used by the search algorithm at the leaves of the search tree to determine which positions are favourable.

The foundation of the evaluation is material balance. Each piece type is assigned a fixed centipawn value: pawns are worth 100, knights 315, bishops 330, rooks 500, queens 900 and kings 20000. The total material score is computed by summing White's piece values and subtracting Black's. This alone produces a functional engine that avoids losing material unnecessarily.

Beyond material, the evaluation includes several positional heuristics to encourage stronger play. Piece-square tables are applied to pawns and knights, assigning a bonus or penalty based on the square a piece occupies. Pawns are rewarded for advancing toward promotion and for occupying central squares, while knights are penalised for sitting on the edges of the board where their mobility is reduced. Rooks receive a bonus for being placed on open or semi-open files, where no pawns obstruct their activity, as well as an additional bonus for reaching the seventh rank, where they can threaten the opponent's back rank. Passed pawns - those with no opposing pawns blocking or guarding their path to promotion - are given a bonus that scales with how far they have advanced.

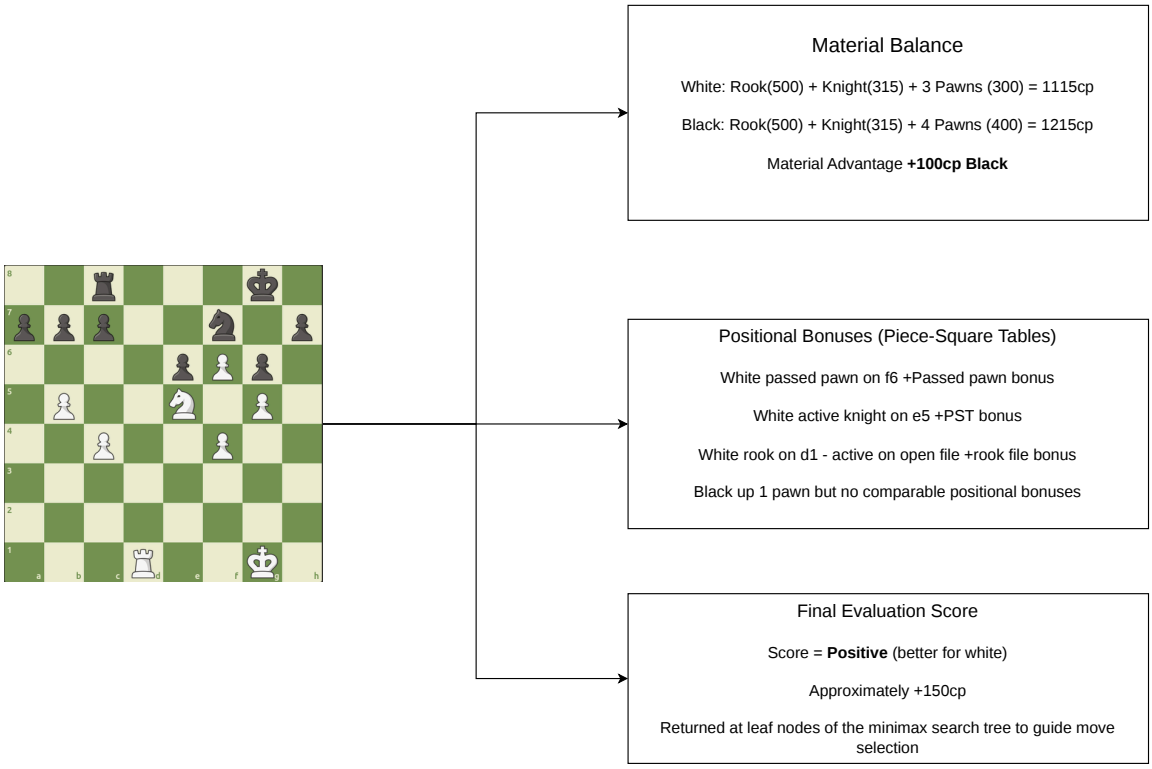


Figure 8: Example evaluation of a board position, showing material balance and positional bonuses contributing to the final score.

Together, these heuristics produce an engine that plays more purposefully than one relying on material counting alone, guiding the search towards positions with better piece activity and long-term structure.

3.1.5 Tuning and Testing

Testing was carried out using a comprehensive suite of unit tests (`/ChessLibrary/src/test/kotlin/EngineTest.kt`) written in Kotlin using the standard `kotlin.test` framework. These tests cover all major components of the engine, from low level board utilities to high-level search behaviour.

Having these tests helped speed up development as it was easy to ensure correctness while making changes to the search and evaluation.

3.2 Human-like move time model

The time a computer opponent takes to reply to a user's moves affects the user experience. While it's usually best to minimise delays or loading times in software applications, a bot that replies near-instantly isn't ideal.

> Paradoxically enough, moving as quickly as possible is not necessarily desirable. Users tend to get frustrated once they realize how little time their Mac really spends to crush them at low levels. In the interest of promoting harmonious Human - Machine relations, we enforce minimum response times (Apple Computer, Inc., 2002).

When creating Mac OS X's Chess application, the developers at Apple added a minimum delay of two seconds before the computer play its moves to avoid frustrating users. A more nuanced approach could improve on this. Human players do not take a uniform two seconds per move: some moves are obvious and are played quickly, while others warrant a longer think. A move time model that reflects this would feel more natural to play against. It could also increase user enjoyment, as a bot taking noticeably longer to reply may give the user the feeling that they have posed a genuine challenge, even if the bot is not truly "stumped".

3.2.1 Data

To train such a model, we used game data from the Lichess.org open database (Lichess.org, 2021). We used the January 2019 dataset, which contains millions of games along with each player's clock times in centisecond precision.

The first 100,000 games in the dataset were filtered to only include those where a clock annotation exists for every move, and where at least four moves were played. Abandoned games and non-standard variants were excluded, as were casual (unrated) games on the basis that they're taken less seriously and may introduce noise. 55.72% of games made it through this filtering.

Moves in each game were converted into rows in our dataset for training the model. Each row contains the following features:

- `initial_time`, the number of seconds on the clock when first starting the game. This feature is the same for all moves within the same game.
- `increment`, the number of seconds added to each player's clock after they make a move. Like `initial_time`, this feature remains the same for all positions within the same game.
- `time_remaining_for_this_position`, the time in seconds remaining on the player's clock for them to make their move.
- `time_opponent_took`, the time in seconds the opponent spent on their previous move.
- `number_of_legal_moves`, the number of legal moves available to the player in the current position.

- `in_check`, whether the player's king is currently in check.
- `was_capture`, whether the move just played by the opponent was a capture.
- `move_number`, the full move number of the current position. This number increases as a game progresses.
- `time_taken_for_this_position`, the target feature. How many seconds the player took to make their move.

Note that since players' clocks don't tick down for their first move on *Lichess.org*, the first 2 moves of every game aren't converted into rows in our training dataset so that we don't have any null values.

In the end, our dataset for training move time models has 3,481,887 rows. This work can be seen in `/EngineMoveTimeModel/src/main/python/PGN to CSV.ipynb`.

For training the linear regression model, two engineered features were included to help model non-linearity in existing features:

- `time_pressure`, defined as $\frac{1}{1+t}$ where t is `time_remaining_for_this_position`, grows sharply as a player's clock runs low. The idea being that players play faster when low on time to avoid losing via timing-out.

- `opening_unfamiliarity`, defined as $\frac{1}{1+e^{6-n}}$ where n is `move_number`. This is a sigmoid that increases from 0 to 1 approximately around the point at which players leave memorised chess territory. This turning point was found empirically through correlation with the target feature to be around move six.

For training a convolutional neural network model, a feature called `pieces` representing the state of the chess board was included. This feature was a 3-dimensional array of booleans of size (12, 8, 8) representing the presence of the 12 possible types of pieces (pawns, knights, bishops, rooks, queens, and kings in both colours) on the 8 by 8 chessboard.

3.2.2 Modelling and results

The dataset was split 80%/10%/10% into training, validation, and test sets. As a baseline, a dummy regressor predicting the mean move time achieved an MSE (mean squared error) of 73.33 on the validation set.

A linear regression model trained on the original features achieved a validation MSE of 56.81. The training data was then cleaned of outliers using the IQR (interquartile range) method, and the two engineered features added, bringing the validation MSE to 62.69. Although this is numerically higher than the unfiltered model, the outlier removal

produced a healthier distribution of predicted move times. Before, there were a notable number of negative predicted move times, which was much improved after outlier removal.

This final linear regression model achieved a test set MSE of 64.40, corresponding to an RMSE (root mean squared error) of approximately 8.0 seconds. The training of this linear regression model can be seen in `/EngineMoveTimeModel/src/main/python/Linear regression model for poster fair (contains was_capture).ipynb`.

We also trained models with CNNs (convolutional neural networks) making use of the board state as an additional feature. The best performing one of these models was trained in `/EngineMoveTimeModel/src/main/python/Keras models using pieces attempt 3 even bigger model with AdamW weight decay.ipynb`. In this model the state of the board, a (12, 8, 8) boolean array, was given to a CNN with 3 hidden layers. The number of filters in the layers increase from 64 to 128 to 512. The first two layers use 3x3 kernels with ReLu (rectified linear unit) activation functions, a standard idea ever since the "VGG" CNNs (Simonyan and Zisserman, 2015). Unlike VGG, we don't use any max-pooling layers because our "image" (the chessboard) is already low resolution (8x8 instead of 224x224). Still, we do get spatial pooling as we don't apply any padding before each filter. The final layer uses 4x4 kernels instead of 3x3 so that each filter gives a single output as opposed to a 2x2 output, reducing dimensionality before the dense layers. These 512 outputs from the CNN were concatenated with the other features and passed through 2 dense layers of 32 neurons each to produce the final output. This overall model had 1,147,617 parameters and achieved a best validation MSE of 51.77 (RMSE $\approx$ 7.2 seconds).

It was surprising that the CNN model was only slightly better than a simple linear regression model (an improvement in RMSE of only 0.8 seconds). A much larger improvement was expected given that the CNN model could make use the state of the board. Instead, we found that the clock situation is the most important factor when it comes to how long a player takes to make their move. The features that correlate most with the target feature are `initial_time`, `time_opponent_took`, `time_remaining_for_this_position`, and `increment`.

Disappointing was the high loss. Even the best model had a RMSE of 7.2 seconds. However, the task we're trying to model doesn't necessarily have a single correct answer. Given the exact same clock and board situation, even the same player may decide to take different amounts of time to move. This results in a high loss no matter the model.

3.2.3 Implementation

Given that the CNN model was large and would have to run as its own microservice while performing only slightly better than linear regression, we chose to implement the linear regression model. This can be seen in `/ChessLibrary/src/main/kotlin/org/`

`checkmateplus/chessLibrary/botEngine/RealisticMoveTimeModel.kt`.

4 Front-end

Checkmate+'s front-end was made in React/Typescript with Vite as a build tool, a common combination. As we use Tailwind, a mobile-first CSS framework, our site works well both on desktop and mobile. Most components on the site are from the ShadCN component library to focus development efforts on the custom chessboard component and its fun animation feature. The visual appearance of the site was overhauled in-between the first and second playtests.

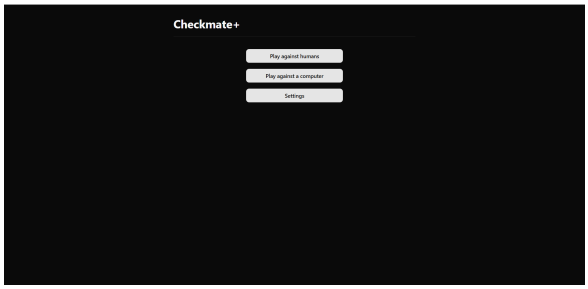


Figure 9: Appearance of main page during the poster fair playtest.

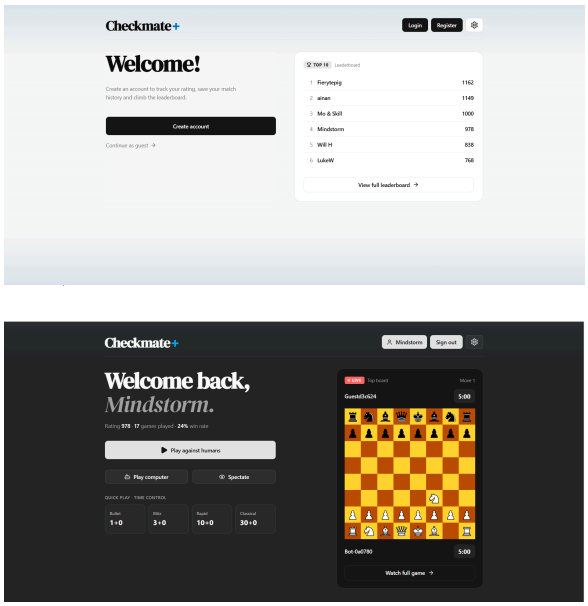


Figure 10: Appearance of main page during the chess club playtest.

4.1 Fun animations

We made the chessboard component ourselves (`/frontend/src/components/ui/ChessBoard.tsx`). The pieces are animated with the help of the Framer Motion animation library. The two animation options we have are "normal" and "fun".

The normal animation option is based on the animations seen on most chess platforms. Piece movement is a linear transition, and a captured piece has an opacity fade out.

The fun animation option adds a rotational wiggle to piece movement, and makes capturing a piece fling it off the board. Neither *Lichess.org* or *Chess.com* offer such an animation option. At most, *Chess.com* offers an option called "Arcade" that adds a blue trail to movement and a red flash to capturing; the pieces themselves don't move any

differently.

4.2 User authentication flow

We decided to only allow authentication via TOTPs (time-based one-time passwords). This avoids having to store users' password hashes, reducing our liability with regard to users' personal information.

When creating an account, a user is guided through our registration flow: first they pick a username, then they add the TOTP secret we provide to their authenticator app of choice (e.g. Google Authenticator, Microsoft Authenticator), and finally they confirm the setup by entering a generated code.

5 Voice commands

Users being able to make moves with their voice is already a feature on other chess platforms. However, our implementation is unique.

Lichess.org implements voice recognition software locally; a user's voice recording never leaves their computer (Lichess.org, 2023). This has benefits in terms of latency but drawbacks in terms of accuracy. As a fallback for when their model gets confused, multiple coloured arrows appear on the user's board. The user then says the colour of the arrow representing their intended move.

Our voice recognition is done in the cloud. We use OpenAI's Whisper to transcribe users' speech snippets. We convert that transcription into a chess move by extracting piece types and square information using regex (regular expressions). The resulting move may not be a legal move, either due to a mistake on our part (transcription) or the user saying an illegal move. So the candidate move is checked for legality with relevant error messages relayed back to the user.

If we are unable to process the move using regex, or if there is ambiguity, we also have a fallback: sending the user's transcription to an LLM (large language model) along with legal moves for context. The LLM is prompted to select the move that best matches the user's transcription. Prompt injection is not a concern as the output is forced to adhere to a certain format (`data class DisambiguationResult`) via OpenAI's structured outputs feature (OpenAI, 2026), and moves are always checked for legality.

This work can be seen in `/VoiceCommandService/src/main/kotlin/org/checkmateplus/voicecommandservice/service`.

6 User enjoyment and feedback

We had two playtest sessions. The first was during the poster fair on 13th March 2026. The second was during a chess club meeting on 13th May 2026.

6.1 Poster fair playtest

The poster fair session limited users to playing against the bot. The move-time model of the bot was randomised to either be our "realistic" model or a "median" model that always made a move after 2.24 seconds. Also randomised was the animation type ("normal" or "fun"). Feedback from this session is in `/User feedback/Poster fair survey.ods`.

Users at the poster fair liked the realistic move-time model. They remarked that they didn't feel rushed and that they can think on the bot's time.

Unfortunately, we didn't get much explicit feedback on the animations. Users' self-reported enjoyment ratings didn't clearly show a preference either: the fun animations averaged 4 out of 5 while the normal animations averaged 4.17. This is insignificant given the small sample size of 9.

Also unfortunate was that the voice command feature was not ready yet during the poster fair, so we couldn't get feedback on it.

The main feedback we received from the poster fair was that it needed to be clearer whose turn it was. We since worked on this by highlighting the squares involved in the most recent move and by making the chess clocks larger and green-coloured when ticking down.

6.2 Chess club playtest

The chess club playtest allowed users to use the site as they pleased. Some users played against the bot and others played against each other. Of note is that all users during this playtest were familiar with chess and online chess platforms. This feedback can be seen in `/User feedback/Chess club playtest feedback.md`.

Many players during this playtest wanted to know the bot's rating. Implementing this without just guessing the bot's rating is feasible and is a point for future development. A possible way could be temporarily sneaking the bot into the ranked human player pool, allowing the bot to lose and win rating from users and establish an accurate rating for itself.

Two players who had just played a game using a real board wanted to use our platform to analyse their game. We didn't have a "free-form analysis" page like other chess platforms have. What the players did instead was match against each other, replay the

moves from the game they wanted to analyse, and then head to the match review. Adding a free-form analysis page is another point for future development.

Related to the analysis, users wanted the match review page to closely align to existing platforms' analysis pages: with evaluation symbols and moves appearing on the board instead of solely in text form within the analysis panel. Users also wanted the option to copy the .pgn (portable game notation) of a game so they could transfer the game data to other sites or software.

Regarding the move-time feature, one user remarked that the bot played too slowly during a "bullet" game: each side only having 1 minute on their clocks. As a result, the user easily won on time. A possible cause for poor performance in quick games is our choice of loss. With MSE, the difference between 3 and 1 seconds is just as erroneous as the difference between 23 and 20 seconds. Relative loss functions like MAPE (mean absolute percentage error) would consider the former example more erroneous, hopefully improving performance in quick games.

The voice command feature was found to be awkward by users. Users quickly abandoned using it in favour moving pieces normally, as for them it was easier and faster. To reduce latency we could make transcription local, only using cloud services for the disambiguation fallback. Users also didn't know they had to click on the button again to stop recording: the button simply said "Listening...". In addition to making the button text clearer, transcribing-as-we-go would allow us to stop the recording automatically once a user has said a move.

Not all users found our account creation flow intuitive. One user needed no guidance at all, while another needed to be told to paste the TOTP secret into their authenticator app. Making this process clearer is another point for future development.

7 Future development

Other than previously mentioned fixes and improvements based on user feedback, there are many features we'd like to add to Checkmate+. The most unique are:

Matchmaking

- Tournament play. An interactive bracket graph, making it easier for both organisers and players.
- Rivals. Highlighting opponents that users have a poor track-record against and encouraging playing against them as a challenge. A counterpart to existing "Friend" systems.

Visuals

- Additional animation options (e.g. pieces that walk, sword fights on captures).
- 3D chess board and customisable pieces. Allowing users to play with 3D characters from their favourite media.

Analysis and bots

- OCR (optical character recognition) tool to convert handwritten scoresheets into reviewable games on our site. Inspired by two players at the chess club wanting to review a game they had written down.
- Personalised neural-network bots. Human-like bots like *Maia* already exist (Tang et al., 2024), but we could offer bots fine-tuned to play like particular users or famous chess players.

8 Conclusions

We set out to investigate the effect of our three features on user enjoyment and instead found that clarity and general ease-of-use was more important. "Whose turn is it?" and "how do I create an account?" were the main pain points for new users, while experienced chess players only found voice commands or move animations to be briefly entertaining gimmicks.

Still, the "realistic bot move-times" feature was well received by most. With improvements to its realism in quicker games, such a feature would be a positive addition to online chess platforms.

References

Apple Computer, Inc. (2002). MBCEngine.mm — Chess source code (v110.3.4), https://web.archive.org/web/20170929075055/https://opensource.apple.com/source/chess/chess-110.3.4/sources/mbcengine.mm. Source file originally dated 2002–2011; archived 2017. Accessed: 2026-05-17.

Chess.com (2025). About chess.com, https://www.chess.com/about. Accessed: 2026-05-16.

Chessprogramming Wiki (2024a). Getting started, https://www.chessprogramming.org/getting_started. Accessed: 2026-11-21.

Chessprogramming Wiki (2024b). Minimax, https://www.chessprogramming.org/minimax. Accessed: 2026-11-22.

Lichess.org (2021). Lichess database, https://database.lichess.org/db-univ/. PGN files with centisecond clock annotations. January 2019 dataset used: https://database.lichess.org/db-univ/lichess_db_univ_2019-01.pgn.bz2. Accessed: 2026-05-17.

Lichess.org (2023). Lichess voice recognition beta, https://lichess.org/@/lichess/blog/lichess-voice-recognition-beta/tqazjvbb. Accessed: 2026-05-16.

OpenAI (2026). Structured outputs, https://developers.openai.com/api/docs/guides/structured-outputs. Accessed: 2026-05-22.

Simonyan, K. and Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. 1409.1556.

Tang, Z., Jiao, D., McIlroy-Young, R., Kleinberg, J., Sen, S. and Anderson, A. (2024). Maia-2: A unified model for human-ai alignment in chess. 2409.20553.

Vrzina, M. (2023). Piece by piece: Building a strong chess engine. BSc Thesis, Vrije Universiteit Amsterdam. Accessed: 2026-12-11.